

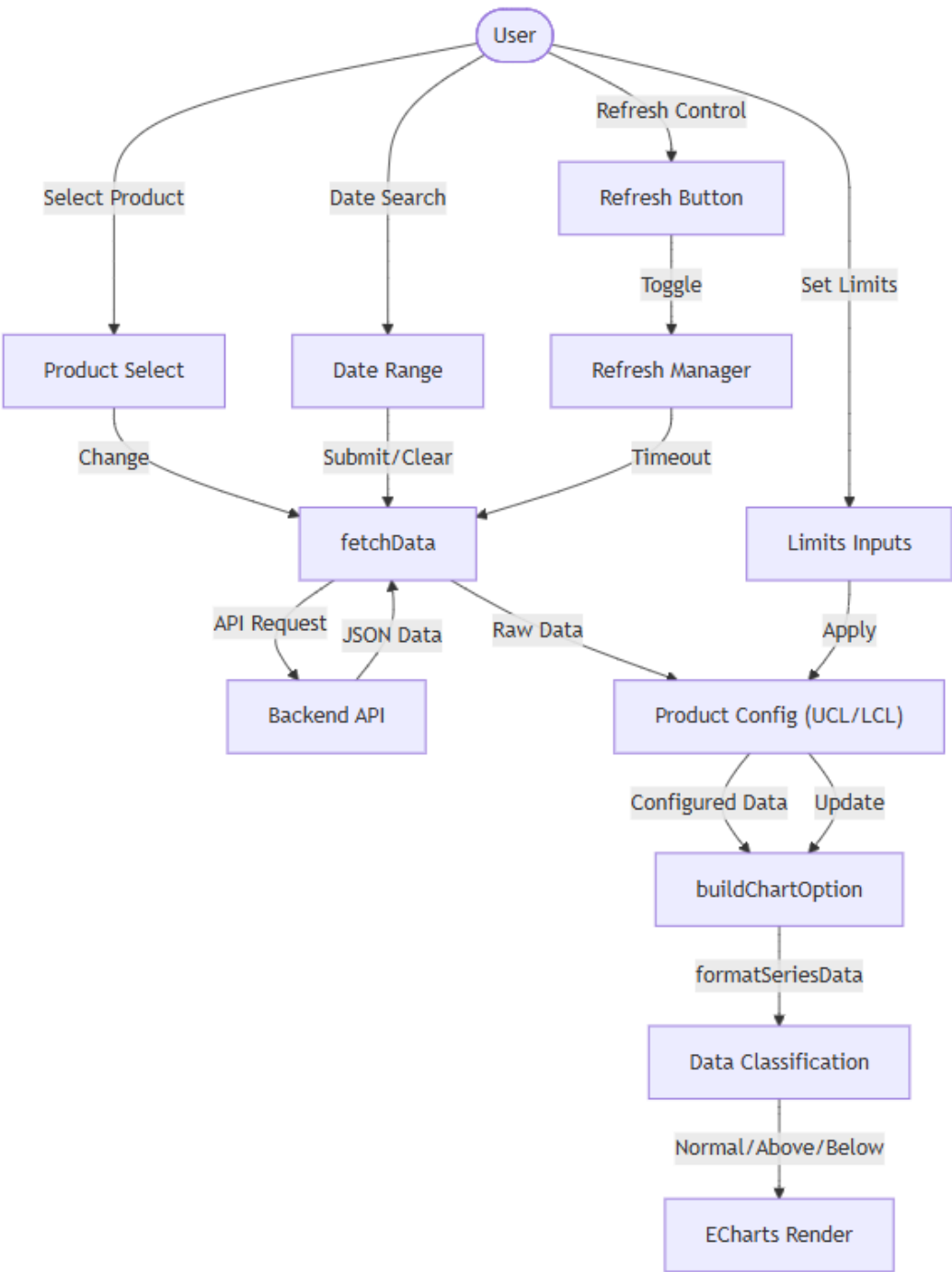
Biscuit Thickness Monitoring Module Logic Analysis

(餅乾厚度監控模組邏輯分析)

1. 系統概觀 (System Overview)

本模組 (`biscuit.js`) 是一個用於監控餅乾厚度數據的前端控制器。它透過 ECharts 呈現即時的厚度散點圖，並根據設定的規格上下限 (UCL/LCL) 將數據分類顯示為不同顏色 (正常、過高、過低)，以利製程人員快速判斷生產品質。

1.1 資料流向圖 (Data Flow)



```
graph TD
    User([User]) -->|Select Product| UI_Product[UI_Product[Product Select]]
    User -->|Set Limits| UI_Limits[UI_Limits[Limits Inputs]]
    User -->|Date Search| UI_Date[UI_Date[Date Range]]
    User -->|Refresh Control| UI_Refresh[UI_Refresh[Refresh Button]]

    UI_Product -->|Change| Logic_Fetch[Logic_Fetch[fetchData]]
    UI_Date -->|Submit/Clear| Logic_Fetch
    UI_Refresh -->|Toggle| Logic_Manager[Logic_Manager[Refresh Manager]]
    Logic_Manager -->|Timeout| Logic_Fetch

    Logic_Fetch -->|API Request| Server[Server[Backend API]]
    Server -->|JSON Data| Logic_Fetch

    Logic_Fetch -->|Raw Data| Logic_Config[Logic_Config["Product Config (UCL/LCL)"]]
    Logic_Config -->|Configured Data| Logic_Chart[Logic_Chart[buildChartOption]]

    Logic_Chart -->|formatSeriesData| Logic_Filter[Logic_Filter[Data Classification]]
    Logic_Filter -->|Normal/Above/Below| Logic_Render[Logic_Render[ECharts Render]]

    UI_Limits -->|Apply| Logic_Config
    Logic_Config -->|Update| Logic_Chart
```

2. 關鍵邏輯詳解 (Key Logic Deep Dive)

2.1 數據獲取 (Data Fetching)

函式: `fetchData(type, dateFrom, dateTo)`

- 流程:

1. 暫停自動刷新管理 (`refreshManager.stop()`)。
2. 構建 API URL (`/biscuit/data`)，附帶 `type` 及可選的 `dateFrom`, `dateTo`。
3. 發送非同步請求，取得 JSON 數據。
4. 配置初始化: 若該產品無預設配置，初始化 `Mean=38`, `Range=6` (`UCL=44`, `LCL=32`)。
5. 呼叫 `buildChartOption` 進行繪圖。
6. 恢復 UI 狀態並重啟自動刷新 (`refreshManager.reset()`)。

2.2 圖表數據分類 (Data Classification)

函式: `formatSeriesData(data, config)` 這是本模組視覺化的核心邏輯。數據不直接繪製，而是先根據 `config.ucl` (上限) 和 `config.lcl` (下限) 進行分類：

- **Normal (藍色):** `lcl <= value <= ucl`
- **Below (橘色):** `value < lcl`
- **Above (紅色):** `value > ucl`

// 邏輯示意

```
normal: data.filter(d => d.sm >= config.lcl && d.sm <= config.ucl)
```

```
below: data.filter(d => d.sm < config.lcl)
above: data.filter(d => d.sm > config.ucl)
```

2.3 圖表建構 (Chart Construction)

函式: `buildChartOption(data, config)`

- **Series:**
 - **Scatter** (散點圖): 分別渲染上述三類數據 (`normal`, `below`, `above`)，使用不同顏色。
 - **Line** (基準線): 繪製 `Mean`, `UCL`, `LCL` 三條參考線 (`markLine`)，輔助判別。
- **X Axis:** 時間軸，格式化為日期/時間。
- **Y Axis:** 厚度數值，固定範圍 0-105。
- **Tooltip:** 客製化顯示 ID、時間與厚度數值。

2.4 自動刷新管理 (Refresh Manager)

物件: `refreshManager`

- 控制倒數計時 (105s) 與自動數據更新。
- 提供 `start`, `stop`, `reset`, `toggle` 方法，與 UI 的暫停/播放按鈕連動。

3. 數據結構定義 (Data Structures)

3.1 產品配置 (Product Config)

```
productConfig = {
  "LL": { // 產品代號
    mean: 38, // 中心值
    ucl: 44, // 管制上限
    lcl: 32 // 管制下限
  }
}
```

3.2 原始數據模型 (Raw Data Model)

後端返回的 `models` 陣列元素：

```
{
  "diecasting_eigenvalue_data_id": Number, // 唯一識別碼
  "dt": String, // ISO 時間字串
  "sm": Number, // 厚度數值
  // ...其他欄位
}
```

4. 事件綁定 (Event Bindings)

- 篩選查詢:
 - `submitBtn`: 驗證日期輸入 -> 呼叫 `fetchData`。
 - `clearBtn`: 清空日期 -> 呼叫 `fetchData` (查全部)。
- 參數設定:
 - `applyLimitsBtn`: 讀取 UI 輸入的 Mean/UCL/LCL -> 更新 `productConfig` -> 重繪圖表。
- 產品切換:
 - `productSelect`: 更新 `currentType` -> `fetchData`。